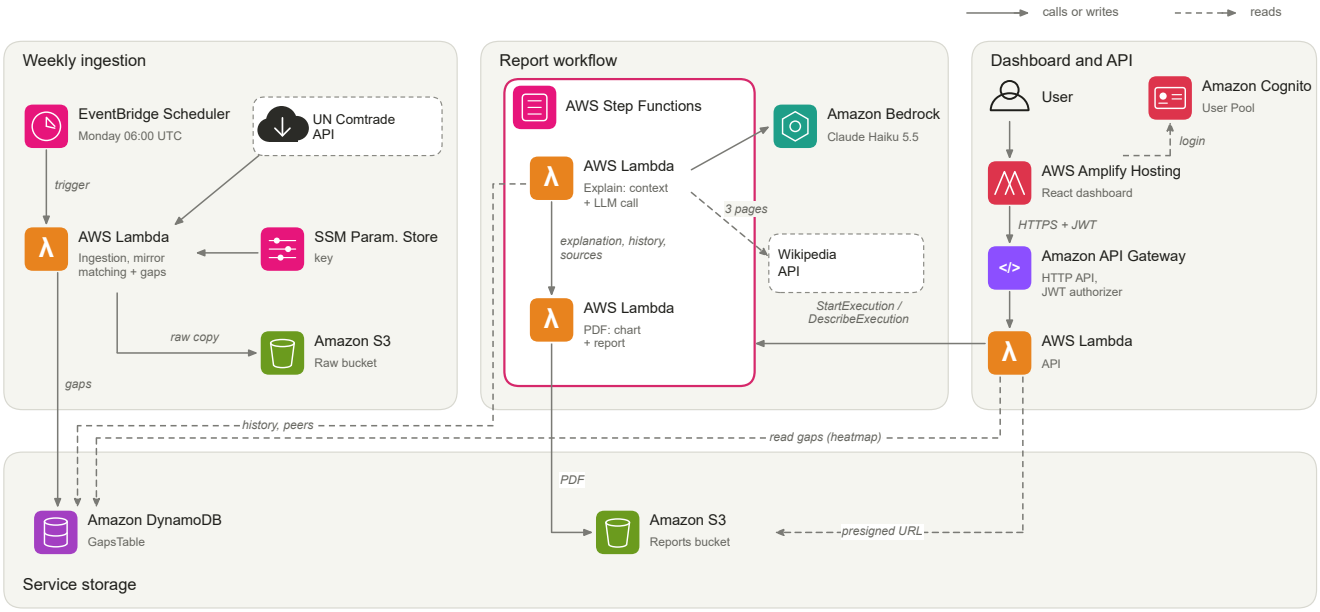
**Project:** Project 3 – Cloud-Native Trade Anomaly Detection & Reporting Platform (client: DNEXT)
**Group:** 16
**Students:** Tania Nunez, Jeremiah Steiner, Aladin Iseni
**Link to a video:** Video presentation (SharePoint, HES-SO)

## Architecture




- **EventBridge Scheduler + Ingestion Lambda:** every Monday, reloads the full history (10 exporters × 7 crops, monthly since 2019) from UN Comtrade, mirror-matches exporter (FOB) and importer (CIF) declarations and flags anomalies (FOB > CIF, gap > 35 %, one-sided flow).
- **S3 Raw bucket:** keeps the raw API answer so that a retried run reuses it instead of calling Comtrade again (fault tolerance).
- **DynamoDB GapsTable:** precomputed gaps per exporter/product/frequency (monthly, quarterly, yearly), read by the dashboard and the report workflow.
- **Amplify + React dashboard:** heatmap of gaps; a red cell launches a report and the UI polls its status until the PDF link appears.
- **Cognito + API Gateway + API Lambda:** JWT-protected HTTP API (`GET /anomalies`, `POST /reports`, `GET /reports/{id}`), documented in OpenAPI.
- **Step Functions report workflow:** two-step asynchronous pipeline with per-step retries; the execution name is the report id, so the same report is never generated twice.
- **Explain Lambda + Bedrock + Wikipedia:** gathers the pair history, peer partners and three Wikipedia pages, then asks Claude for a schema-validated explanation (hypotheses, checks, sources).
- **PDF Lambda + S3 Reports bucket:** renders the history chart and the PDF report, stored privately and downloaded through a presigned URL.

## Deployment

The whole backend is described as Infrastructure as Code in a single **AWS SAM** template (`backend/template.yaml`) and deployed in `us-east-1` as the CloudFormation stack `group16-trade-anomaly`; the front-end is deployed separately on Amplify. The solution is fully serverless and billed per use, since the load is low and irregular (weekly data, on-demand reports).

1. **Functions (FaaS):** four AWS Lambda functions in Python 3.14 – *Ingestion* (900 s, 2 GB; a measured full reload took 370 s, within the 15-minute Lambda limit), *API* (30 s), *Explain* (300 s, 1 GB), *PDF* (120 s, 1 GB).
2. **Orchestration:** AWS Step Functions *standard* state machine (`Explain` → `BuildPdf`), defined in Amazon States Language; only a few KB pass between steps, well below the 256 KB limit.
3. **Database:** Amazon DynamoDB, on-demand capacity, composite key `exporter#hs4#freq` / `period#partner`.
4. **Object storage:** two Amazon S3 buckets with all public access blocked (raw Comtrade data, PDF reports).
5. **AI service:** Amazon Bedrock, model Claude Haiku 5.5 through the global inference profile, called with `InvokeModel` under the Lambda IAM role (no key in the code).
6. **API and security:** Amazon API Gateway HTTP API with a Cognito JWT authorizer and CORS limited to the Amplify domain; accounts created by an administrator only; Comtrade key in SSM Parameter Store (SecureString).
7. **Front-end:** AWS Amplify Hosting builds the React/Vite app from the Git repository (`amplify.yml`, monorepo root `frontend/`).
8. **Scheduling:** Amazon EventBridge Scheduler triggers the weekly reload (`cron(0 6 ? * MON *)`).

## Tasks

| Tasks | Outcome | Students involved |
|---|---|---|
| Comparative study of cloud services | Serverless choice justified service by service, with the rejected alternative (Glue, ECS Fargate, direct Redshift reads, ...) | All three |
| Design of the architecture | Architecture document with overview and detailed diagrams (3 flows: ingestion, dashboard, report) | All three |
| Data ingestion pipeline | Weekly Comtrade ingestion with raw S3 copy, retries and SSM-stored key | Jeremiah Steiner |
| Mirror matching and anomaly rule | Gap computation with edge cases (zero FOB, one-sided flows, quarterly/yearly aggregation), DynamoDB data model, unit tests | Jeremiah Steiner |
| Dashboard and hosting | React heatmap, report panel with status polling, deployment on Amplify Hosting | Tania Nunez |
| API and authentication | HTTP API with three routes, OpenAPI specification, Cognito user pool and JWT authorizer | Tania Nunez |
| Report workflow (Step Functions) | State machine `Explain` → `BuildPdf` with retry policies, 256 KB payload constraint, idempotent report ids | Aladin Iseni |
| Generative AI layer | Bedrock call with structured output, Wikipedia context retrieval, PDF report with chart and sources | Aladin Iseni |
| Final deployment with AWS SAM | Complete SAM template (IAM policies, parameters, outputs), `samconfig.toml`, deployment guide and budget alert | Aladin Iseni |
| Security and cost | Least-privilege IAM, private buckets, presigned URLs, pay-per-use services, limits documented (15 min Lambda) | All three |